

Pamantasan ng Lungsod ng Maynila

College of Information Systems and Technology Management

System Development Plan [CONFIDENTIAL]

Development of an Integrated Information System for National Service Training Program of Pamantasan ng Lungsod ng Maynila with Attendance, Engagement, and Performance Monitoring

Java SE 21 | Java Swing | Famerio – Capulong – Tinoko – Vallite – De Leon - Baterbonia

Project	NSTP Integrated Information System
Platform	Java SE 21 + Java Swing (Desktop Application)
Database	MySQL via JDBC (XAMPP / MySQL Server, local hosting)
Group	GROUP FIFTH UNIT
Instructor	Prof. Darwin C. Co
Date	May 2026

Phase Overview

Phase	Description
Phase 1 — Days 1-2	Initial Setup: environment, MySQL, JDBC, project structure
Phase 2 — Day 3	Database Design and Integration: schema, tables, JDBC connection test
Phase 3 — Day 4	Core System Structure: packages, session management, RBAC
Phase 4 — Day 5	GUI Foundation: login screen, main frame, navigation shell
Phase 5 — Days 6-10	Module Development: all 7 system modules built one by one
Phase 6 — Day 11	System Integration: connect all modules, role access, data flow
Phase 7 — Day 12	Testing and Debugging: unit tests, error fixes, data validation
Phase 8 — Days 13-14	Finalization: UI polish, final tests, demo preparation

MAIN OBJECTIVE

Set up the complete development environment, install all required tools, create the Java project with the correct structure, and confirm that every team member has the same working setup before any code is written.

DETAILED TASKS**Task 1.1 — Install Required Software**

1. Download and install the Java Development Kit (JDK) version 21 from the official Oracle or OpenJDK website. Confirm the installation by running 'java -version' in the terminal or command prompt and verifying that it returns version 21.
2. Download and install Apache NetBeans IDE (latest stable release) or IntelliJ IDEA Community Edition. This will be the primary development environment for the entire project.
3. Download and install XAMPP from apachefriends.org. This provides the local MySQL Server and phpMyAdmin, which will be used to manage the database throughout development.
4. Download the MySQL Connector for Java (JDBC driver) JAR file, version 8.0 or higher, from the MySQL downloads page. Save this file to a shared project folder that all team members can access.

Task 1.2 — Configure the IDE

5. Open the IDE and create a new Java project. Name the project 'NSTP_InfoSystem'. Set the project location to a local folder that is also backed up to a shared drive or version control system.
6. Add the MySQL Connector JAR file as an external library in the project settings. In NetBeans, right-click the project, select Properties, then Libraries, then add the JAR file. In IntelliJ, go to Project Structure, then Modules, then Dependencies.
7. Confirm that the IDE recognizes the MySQL Connector by creating a temporary empty class and importing 'java.sql.Connection'. The IDE should not show an import error.

Task 1.3 — Start XAMPP and Verify MySQL

8. Open the XAMPP Control Panel and start the Apache and MySQL services. Confirm that both show a green status indicator.
9. Open a browser and navigate to 'localhost/phpmyadmin'. Confirm that phpMyAdmin loads correctly. This confirms that the MySQL server is running and accessible.
10. Note the default MySQL root password (blank by default in XAMPP). If the team decides to set a password, all members must record it in a shared project notes document.

EXPECTED OUTPUT

- JDK 21 installed and verified on all team machines.
- NetBeans or IntelliJ installed and project 'NSTP_InfoSystem' created.
- MySQL Connector JAR added to the project as an external library.
- XAMPP running with MySQL and phpMyAdmin accessible at localhost.

COMMON ISSUE: If the IDE shows a 'Cannot resolve symbol' error for java.sql.Connection even after adding the JAR, close and reopen the IDE, then re-check the Libraries section in Project Properties to confirm the JAR path is still correctly linked.

TIP: Save the XAMPP installer and MySQL Connector JAR to a shared USB or cloud folder. This prevents delays if a team member needs to reconfigure their machine.

DAY 2

PHASE 1 — INITIAL SETUP

Database Creation, Schema Design, and JDBC Connection

MAIN OBJECTIVE

Create the MySQL database, define all required tables with proper columns and data types, and establish a working JDBC connection from the Java project to the MySQL database.

DETAILED TASKS

Task 2.1 — Create the Database in phpMyAdmin

11. Open phpMyAdmin at localhost/phpmyadmin. Click 'New' on the left panel. Name the database 'nstp_infosystem'. Set the collation to 'utf8mb4_general_ci' for full character support. Click Create.
12. Verify that the database 'nstp_infosystem' appears in the left panel of phpMyAdmin.

Task 2.2 — Create All Database Tables – This Will Be The Basis of the CHAPTER 3 PART “ENTITY RELATIONSHIP DIAGRAM”

Create the following tables inside the 'nstp_infosystem' database. Use phpMyAdmin's table creation wizard or run the SQL statements in the phpMyAdmin SQL tab.

Table Name	Columns and Data Types
users	user_id (INT, PK, AUTO_INCREMENT), username (VARCHAR 100), password_hash (VARCHAR 255), role (ENUM: Admin/CWTS_Student/ROTC_Student/ROTC_Senior), student_id (INT, FK), is_active (TINYINT)
students	student_id (INT, PK, AUTO_INCREMENT), full_name (VARCHAR 150), program (ENUM: CWTS/ROTC), year_level (VARCHAR 20), section (VARCHAR 20), contact_no (VARCHAR 20), email (VARCHAR 100)
attendance	record_id (INT, PK, AUTO_INCREMENT), student_id (INT, FK), session_name (VARCHAR 100), session_date (DATE), status (ENUM: Present/Absent/Late/Excused), remarks (TEXT)
engagement	activity_id (INT, PK, AUTO_INCREMENT), student_id (INT, FK), activity_name (VARCHAR 150), activity_date (DATE), location (VARCHAR 150), category (VARCHAR 80), hours_rendered (DECIMAL 5,2), verified (TINYINT)
performance	entry_id (INT, PK, AUTO_INCREMENT), student_id (INT, FK), drill_name (VARCHAR 100), drill_date (DATE), rating (ENUM: Excellent/Proficient/Satisfactory/Needs_Improvement/Unsatisfactory), remarks (TEXT), officer_name (VARCHAR 100)

points	transaction_id (INT, PK, AUTO_INCREMENT), student_id (INT, FK), amount (INT), type (ENUM: award/deduct), reason (TEXT), transaction_date (DATE), admin_id (INT)
demerits	demerit_id (INT, PK, AUTO_INCREMENT), student_id (INT, FK), reason (VARCHAR 200), severity (ENUM: Minor/Moderate/Major), notes (TEXT), officer_name (VARCHAR 100), demerit_date (DATE), status (ENUM: Active/Appealed/Dismissed)
announcements	announcement_id (INT, PK, AUTO_INCREMENT), title (VARCHAR 200), body (TEXT), audience (ENUM: All/CWTS/ROTC), priority (ENUM: Normal/Important/Urgent), posted_date (DATETIME), posted_by (INT), is_pinned (TINYINT)
notifications	notif_id (INT, PK, AUTO_INCREMENT), student_id (INT, FK), type (VARCHAR 80), message (TEXT), created_date (DATETIME), is_read (TINYINT)
disputes	dispute_id (INT, PK, AUTO_INCREMENT), student_id (INT, FK), record_id (INT, FK), reason (TEXT), requested_status (ENUM), submitted_date (DATE), status (ENUM: Pending/Approved/Rejected), admin_response (TEXT)

- After creating all tables, verify each one by clicking on it in phpMyAdmin and checking that all columns are defined correctly. Export the full database as a .sql file and save it to the shared project folder as a backup.

Task 2.3 — Create a Java Database Utility Class

- Inside the Java project, create a new package named 'util'. Inside this package, create a class named 'DBConnection'. This class will hold the database connection logic used by all modules.
- Write a static method named 'getConnection()' in DBConnection that uses DriverManager.getConnection() with the JDBC URL 'jdbc:mysql://localhost:3306/nstp_infosystem', username 'root', and the MySQL password (blank for default XAMPP). The method should return a Connection object.
- Write a static method named 'closeConnection(Connection conn)' that checks if the connection is not null and not already closed, then calls conn.close().
- In a temporary test class, call DBConnection.getConnection() and print a success message to the console if the connection is established, or print the error message if it fails. Run this test and confirm the output reads 'Database connection successful.'

EXPECTED OUTPUT

- Database 'nstp_infosystem' created in MySQL with all 10 tables verified.
- Full database schema exported as a .sql backup file.
- DBConnection utility class written and tested.
- Console output confirms a successful live JDBC connection to MySQL.

COMMON ISSUE: If the JDBC connection fails with 'Communications link failure,' confirm that the MySQL service in XAMPP is running. Also check that the port is 3306 (default) and that the database name in the JDBC URL matches exactly.

COMMON ISSUE: If you receive 'Access denied for user root,' check the MySQL password. In XAMPP, the default root password is blank. If someone set a password previously, use that value in the connection string.

TIP: Insert one sample row into the students table using phpMyAdmin after the tables are created. Use this row for testing JDBC queries on Day 4 and Day 5.

DAY

3

PHASE 2 — CORE SYSTEM STRUCTURE

Project Structure, Session Management, and RBAC Foundation

MAIN OBJECTIVE

Organize the Java project into a clean package structure, implement the session management class that tracks the logged-in user, and build the foundation of the role-based access control system that will control which modules each user type can access.

DETAILED TASKS

Task 3.1 — Define the Project Package Structure

18. Create the following packages inside the Java project. Each package will hold a specific category of classes throughout the entire project.

- ui.login — Login and authentication screens
- ui.admin — Admin dashboard and admin-only panels
- ui.student — Student dashboards for CWTS and ROTC
- ui.shared — Panels used by multiple roles (calendar, announcements)
- model — Plain data model classes (User, Student, AttendanceRecord, etc.)
- dao — Data Access Object classes that handle all SQL queries per module
- service — Business logic classes (rate computation, flagging, report assembly)
- util — Utility classes (DBConnection, DateHelper, FileExporter)
- session — Session state management class

Task 3.2 — Create the Session Manager

19. Inside the 'session' package, create a class named 'SessionManager'. This class must use the Singleton design pattern so that only one instance exists at any time during the application's run.
20. Add the following fields to SessionManager: currentUserId (int), currentUsername (String), currentRole (String), currentStudentId (int), isLoggedIn (boolean).
21. Write a static method 'getInstance()' that returns the single instance. Write getter and setter methods for each field. Write a 'clearSession()' method that resets all fields to their default null or false values. This method will be called on logout.

Task 3.3 — Create Model Classes

22. Inside the 'model' package, create the following plain Java classes. Each class should have private fields, a constructor, and public getter and setter methods. Do not add database logic to model classes.
 - User — fields: userId, username, passwordHash, role, studentId, isActive
 - Student — fields: studentId, fullName, program, yearLevel, section, contactNo, email
 - AttendanceRecord — fields: recordId, studentId, sessionName, sessionDate, status, remarks

- EngagementActivity — fields: activityId, studentId, activityName, activityDate, location, category, hoursRendered, verified
- PerformanceEntry — fields: entryId, studentId, drillName, drillDate, rating, remarks, officerName
- PointTransaction — fields: transactionId, studentId, amount, type, reason, transactionDate, adminId
- Demerit — fields: demeritId, studentId, reason, severity, notes, officerName, demeritDate, status

Task 3.4 — Build the Application Entry Point

23. Create a class named 'Main' in the root package. The main method should call `SwingUtilities.invokeLater()` and inside it, create an instance of the `LoginFrame` (which will be built on Day 5) and make it visible. For now, use a placeholder `System.out.println('System started')` until the `LoginFrame` is ready.

EXPECTED OUTPUT

- All packages created and visible in the project structure.
- SessionManager class implemented as a Singleton with all required fields and methods.
- All seven model classes created with fields, constructors, and getters/setters.
- Main entry point class created and functional.

MILESTONE 1 — End of Day 3: The project has a clean package structure, a working JDBC connection, a live MySQL database with all tables, and the session and model foundation. The team is ready to begin building the user interface.

TIP: Assign one team member to maintain the `DBConnection` and `SessionManager` classes throughout the project. Changes to these classes affect every module, so having one owner avoids conflicting edits.

DAY
4

PHASE 3 — GUI FOUNDATION

Login Interface, Authentication Logic, and Main Frame Shell

MAIN OBJECTIVE

Build the complete login screen with authentication logic connected to the MySQL users table, and create the main application window shell that will host all module panels through a card layout and sidebar navigation.

DETAILED TASKS

Task 4.1 — Create the LoginFrame

24. In the 'ui.login' package, create a class named 'LoginFrame' that extends `JFrame`. Set the title to 'NSTP Information System', set the size to 500x400, set the default close operation to `EXIT_ON_CLOSE`, and center the frame on the screen using `setLocationRelativeTo(null)`.
25. Add the following components using a `JPanel` with `GridBagLayout`: a school logo or header label at the top, a `JLabel` and `JTextField` for username input, a `JLabel` and `JPasswordField` for password input, a `JCheckBox` labeled 'Remember Me', a 'Login' `JButton`, and a 'Forgot Password' `JLabel` styled as a clickable link.

26. Apply basic styling: set the background color of the main panel to a dark navy color, use white or gold colored text for labels, and apply a consistent font (Arial, 12pt) to all components.

Task 4.2 — Create the User DAO and Authentication Logic

27. In the 'dao' package, create a class named 'UserDAO'. Write a method named 'validateLogin(String username, String password)' that queries the users table using a PreparedStatement. The query should check for a matching username and is_active = 1. Return the User object if found, or null if not found.
28. In the LoginFrame, add an ActionListener to the Login button. When clicked, it should call UserDAO.validateLogin() with the entered credentials. If the result is not null, verify the password against the stored hash. If the password matches, store the user data in SessionManager and open the appropriate dashboard based on the role. If login fails, show a JOptionPane error dialog.
29. Implement a basic failed-attempt counter. After 3 failed attempts in the same session, disable the Login button and show a message that the account is temporarily locked. This is a UI-level check only and does not require database changes at this stage.

Task 4.3 — Create the Main Application Frame Shell

30. In the 'ui.admin' package, create a class named 'MainFrame' that extends JFrame. This frame will be shared by the Admin and ROTC Senior roles. Set its size to 1200x700 and center it on screen.
31. Divide the frame using BorderLayout. The West region will hold a collapsible sidebar JPanel. The Center region will hold a CardLayout JPanel that displays the active module. The North region will hold a header JPanel showing the user's name, role, and the current date.
32. Add placeholder navigation buttons to the sidebar: Dashboard, Attendance, Engagement, Performance, Points, Demerits, Calendar, Announcements, Reports, and Logout. These buttons will be wired to their panels as each module is built. For now, clicking any button should print the button name to the console.
33. Create a separate 'StudentFrame' class in the 'ui.student' package using the same structure but with a sidebar tailored for student-accessible modules only: Dashboard, Attendance, Engagement or Performance, Points, Notifications, and Logout.

EXPECTED OUTPUT

- Fully functional login screen with username, password, and Remember Me components.
- UserDAO with validateLogin method connected to the MySQL users table.
- Login correctly routes to MainFrame or StudentFrame based on user role.
- MainFrame and StudentFrame shells with sidebar navigation and CardLayout center panel.
- Header panel showing current user name and role.

COMMON ISSUE: If the GUI appears but the layout looks broken or components overlap, check that you are calling pack() or setSize() after adding all components and that setVisible(true) is called inside SwingUtilities.invokeLater().

TIP: Insert one test admin account directly into the users table using phpMyAdmin before testing login. Use username 'admin', set role to 'Admin', and store a plain text password for now. Password hashing can be added after the basic login flow works.

MAIN OBJECTIVE

Build the complete User Management module for the Administrator, including the ability to add, edit, deactivate, and view all user accounts. Also build the Student Dashboard panel that displays a summary of key records for each logged-in student.

DETAILED TASKS**Task 5.1 — User Management Panel (Admin)**

34. In the 'ui.admin' package, create a panel class named 'UserManagementPanel' that extends JPanel. This panel will be loaded into the MainFrame's CardLayout when the admin navigates to user management.
35. Add a JTable at the top half of the panel to display all users. The table should show columns: User ID, Username, Role, Linked Student Name, and Status (Active or Inactive). Use a DefaultTableModel and populate it by calling a new UserDao method named 'getAllUsers()' that runs a SELECT query joining the users and students tables.
36. Add a button panel below the table with three buttons: Add User, Edit User, and Deactivate User. Add User and Edit User should open a JDialog form with fields for username, password, role, and linked student selection. Deactivate User should update the is_active field for the selected row to 0.
37. Write the corresponding DAO methods: 'addUser(User user)' using an INSERT PreparedStatement, 'updateUser(User user)' using an UPDATE PreparedStatement, and 'deactivateUser(int userId)' using an UPDATE PreparedStatement that sets is_active to 0.
38. Add a live search JTextField above the table. Use a DocumentListener to filter rows in the DefaultTableModel on every keystroke by student name or username.

Task 5.2 — Student Dashboard Panel

39. In the 'ui.student' package, create a panel class named 'StudentDashboardPanel' that extends JPanel. This panel is the first screen a student sees after login.
40. Load the logged-in student's data from SessionManager. Query the relevant tables to populate summary cards: attendance rate (computed from the attendance table), current points balance (sum of transactions from the points table), engagement hours total (sum from the engagement table, CWTS only), and active demerit count (count from the demerits table, ROTC only).
41. Design the summary cards as styled JPanels using GridLayout. Each card should show a label, a large value, and a color-coded status indicator. For example, attendance below 75 percent shows red, and above 75 percent shows green.

EXPECTED OUTPUT

- User Management panel fully functional: displays all users in JTable, add/edit/deactivate working with MySQL.
- Student Dashboard displays accurate summary data retrieved from MySQL for the logged-in student.
- All DAO methods for user management written and tested.

MILESTONE 2 — End of Day 5: The login, main frames, user management, and student dashboard are all functional. The team now has a working entry point into the system and can begin building the main functional modules.

DAY
6

PHASE 4 — MODULE DEVELOPMENT

Attendance Module (Admin Logging and Student View)

MAIN OBJECTIVE

Build the complete Attendance module including the administrator session logging interface, the student-facing attendance tracker view, and the dispute submission form. All data must be stored and retrieved from the MySQL attendance table.

DETAILED TASKS

Task 6.1 — Admin Attendance Logging Panel

42. Create a panel class named 'AttendanceAdminPanel'. Add a session name JTextField, a session date JSpinner using SpinnerDateModel, and a JTable that lists all enrolled students with a JComboBox status selector in each row set to Present, Absent, Late, or Excused.
43. Add bulk action buttons: 'Mark All Present' and 'Mark All Absent'. These should apply the selected status to every row in the table at once.
44. Before saving, check whether an attendance record for the same session name and date already exists in the database. If it does, show a warning JDialog asking the admin to confirm before overwriting. This duplicate-session guard prevents accidental double entries.
45. Write a DAO method named 'saveAttendanceSession(String sessionName, Date sessionDate, List records)' that uses a batch INSERT or UPDATE PreparedStatement to write all records to the attendance table in a single database operation.
46. After saving, call a service method 'computeAttendanceRate(int studentId)' that queries the attendance table for the student, counts the total records and the number of Present records, and returns a percentage. Store the result and use it to update the student dashboard on next load.

Task 6.2 — Student Attendance Tracker Panel

47. Create a panel class named 'AttendanceStudentPanel'. Load all attendance records for the logged-in student from the attendance table using a DAO method named 'getAttendanceByStudent(int studentId)'.
48. Display the records in a color-coded JTable: Present rows in light green, Absent rows in light red, Late rows in light yellow, and Excused rows in light blue. Show a summary above the table with total counts and the computed attendance rate with a JProgressBar.
49. When the student double-clicks an Absent or Late row, show a button labeled 'Submit Dispute'. Clicking it opens a JDialog with a reason JTextArea and a dropdown for the requested correction. The submission should be saved to the disputes table via a DAO method named 'submitDispute(Dispute dispute)'.

EXPECTED OUTPUT

- Admin can log attendance for all students in a session and save records to MySQL.
- Duplicate session guard prevents overwriting without confirmation.

- Student attendance tracker loads personal records with correct color coding.
- Attendance rate correctly computed from database records.
- Dispute submission form saves entries to the disputes table.

COMMON ISSUE: If the JTable does not update after saving, call 'tableModel.fireTableDataChanged()' or rebuild the DefaultTableModel after the save operation to force a visual refresh.

TIP: Use PreparedStatement with batch execution (addBatch and executeBatch) when saving attendance for a full class. This is much faster than running one INSERT per student in a loop.

DAY 7

PHASE 4 — MODULE DEVELOPMENT

Engagement Module (CWTS) and Performance Module (ROTC)

MAIN OBJECTIVE

Build the CWTS Engagement module that logs and tracks community service activities and hours, and the ROTC Performance module that logs drill ratings and computes average performance scores. Both modules must be fully connected to MySQL.

DETAILED TASKS

Task 7.1 — Engagement Module (CWTS)

50. Create a panel named 'EngagementAdminPanel'. Add an activity entry form with fields for activity name, category (JComboBox), date, location, description, and hours rendered. Add a student selector that allows the admin to assign the activity to one student or to multiple students using a JList with checkboxes.
51. Write a DAO method named 'addEngagementActivity(EngagementActivity activity)' that inserts the record into the engagement table. For bulk assignment, loop through the selected students and call this method for each one within the same database transaction.
52. Create a panel named 'EngagementStudentPanel'. Load all engagement records for the logged-in CWTS student. Display them in a JTable with columns for activity name, date, hours, and verification status. Show the total hours rendered in a large label below the table and a JProgressBar indicating progress toward the required minimum (default 20 hours).
53. Write a service method named 'isEngagementComplete(int studentId)' that queries the engagement table for the student, sums the hours_rendered where verified = 1, and compares the result against the configured minimum. Return a boolean that controls whether the completion badge displays on the student view.

Task 7.2 — Performance Module (ROTC)

54. Create a panel named 'PerformanceAdminPanel'. Add a drill entry form with fields for drill name, date, rating (JComboBox with all five levels), and remarks. Add a student selector for individual or bulk rating entry.
55. Write a DAO method named 'addPerformanceEntry(PerformanceEntry entry)' that inserts the record into the performance table. Write a separate method named 'getAverageRating(int studentId)' that queries the performance table and computes the average numerical equivalent of the rating enum values for a given student.

56. Write a service method named 'checkPerformanceFlag(int studentId)' that retrieves the last three performance entries for the student ordered by date descending, checks if all three are Needs Improvement or Unsatisfactory, and returns true if the at-risk flag should be activated.
57. Create a panel named 'PerformanceStudentPanel'. Display the student's drill history in a color-coded JTable where Excellent is dark green, Proficient is green, Satisfactory is yellow, Needs Improvement is orange, and Unsatisfactory is red. Show the computed average rating and the at-risk flag if applicable.

EXPECTED OUTPUT

- Engagement module logs CWTS activities to MySQL with bulk assignment support.
- Engagement student view shows total hours and completion progress bar.
- Performance module logs ROTC drill ratings to MySQL.
- Average rating and at-risk detection logic working correctly.
- Performance student view shows color-coded history.

COMMON ISSUE: If the hours total does not update immediately after adding a new activity, confirm that your student panel reloads its data by re-running the DAO query each time the panel is opened or a save event fires.

TIP: Map rating enum values to integers (Excellent=5, Proficient=4, Satisfactory=3, Needs Improvement=2, Unsatisfactory=1) for the average calculation. Store this mapping in a helper class in the util package.

DAY 8

PHASE 4 — MODULE DEVELOPMENT

Points Module and Demerit Module

MAIN OBJECTIVE

Build the Points tracking module for both CWTS and ROTC students, the Points Leaderboard panel, and the Demerit management module for ROTC. All data must be persisted in and retrieved from MySQL.

DETAILED TASKS

Task 8.1 — Points Module

58. Create a panel named 'PointsAdminPanel'. Add separate forms for awarding and deducting points. Both forms should include a student selector JComboBox, an amount JSpinner, a reason JTextField, and a date JSpinner. The deduction form should require a justification text area that must not be empty before saving.
59. Write DAO methods: 'addPointTransaction(PointTransaction transaction)' that inserts a record into the points table, and 'getPointsBalance(int studentId)' that runs a SUM query on the points table for the student, adding award amounts and subtracting deduction amounts to return the net balance as an integer.
60. Add a negative balance protection check. Before saving a deduction, call getPointsBalance() and check whether the balance after deduction would be negative. If so, show a warning dialog and do not save.

61. Create a panel named 'PointsStudentPanel'. Show the current net balance in a large bold label at the top with a JProgressBar below it. Display the full transaction history in a JTable with color-coded amount column: green for awards and red for deductions. Add a filter by month using JComboBox.
62. Create a panel named 'LeaderboardPanel' accessible to all users. Run a query that groups points transactions by student_id, computes the net balance for each student separately for CWTS and ROTC, and orders the results descending. Display the top 10 in two separate JTables with rank, name, and balance. Highlight the logged-in student's row in light blue.

Task 8.2 — Demerit Module

63. Create a panel named 'DemeritAdminPanel' accessible to the Administrator and ROTC Senior Officer. Add a demerit assignment form with fields for student selector (ROTC only), reason JComboBox, severity JComboBox (Minor, Moderate, Major), notes JTextArea, and date JSpinner. Show a confirmation dialog before saving.
64. Write DAO methods: 'addDemerit(Demerit demerit)' for insert, 'dismissDemerit(int demeritId, String justification)' for updating the status to Dismissed, and 'getDemeritsForStudent(int studentId)' for retrieving all records for a student.
65. Create a panel named 'DemeritStudentPanel'. Display the student's demerit history in a color-coded JTable (Minor=yellow, Moderate=orange, Major=red). Show the total active demerit count and a 'Safe Standing' or 'Demerit Warning' badge based on a configurable threshold.

EXPECTED OUTPUT

- Admin can award and deduct points with negative balance protection.
- Points student view shows balance, progress bar, and transaction history.
- Leaderboard shows top 10 students per program with correct ranking.
- Admin and ROTC Senior can assign demerits saved to MySQL.
- Student demerit view shows color-coded history and standing badge.

COMMON ISSUE: If the leaderboard loads slowly, check that the SQL query uses a SUM with GROUP BY rather than fetching all individual records into Java and computing the sum in a loop. Database-level aggregation is always faster.

TIP: For the severity weight in demerit scoring (Minor=1, Moderate=2, Major=3), compute the weighted total in a SQL query using CASE WHEN expressions rather than doing it in Java. This keeps the logic consistent and faster.

DAY
9

PHASE 4 — MODULE DEVELOPMENT

Communication Module: Announcements, Calendar, and Notifications

MAIN OBJECTIVE

Build the Announcements board, the Activity Calendar, and the Student Notification Center. All three communication features must be connected to MySQL and must apply audience filtering so that CWTS and ROTC students only see content relevant to their program.

DETAILED TASKS

Task 9.1 — Announcements Module

66. Create a panel named 'AnnouncementAdminPanel'. Add a post announcement form with fields for title, body JTextArea, audience JComboBox (All, CWTS Only, ROTC Only), priority JComboBox (Normal, Important, Urgent), and a pin JCheckBox. Write a DAO method 'postAnnouncement(Announcement a)' that inserts the record into the announcements table.
67. Create a panel named 'AnnouncementStudentPanel'. Query the announcements table filtering by audience matching the logged-in student's program (fetch 'All' records and program-specific records). Display each announcement as a styled card in a scrollable JPanel using BoxLayout. Pinned announcements appear at the top. Urgent announcements show a red left-border accent. Unread announcements show a 'NEW' badge.
68. Track read status by storing a separate table or using a read_announcements linking table that records which student has read which announcement. When a student opens an announcement card, insert a read record into this table. The 'NEW' badge disappears on next load.

Task 9.2 — Activity Calendar Module

69. Create a panel named 'CalendarPanel' accessible to all roles. Render a monthly calendar as a JPanel with GridLayout(7,7): one row for day-name headers and six rows for date cells. Previous and Next JButtons update the displayed month.
70. Load all calendar events from the database filtered by the current user's program. Display a colored dot on any date cell that has an event (blue for CWTS, red for ROTC, gold for General). Clicking a date cell opens a JDialog listing all events for that day.
71. Create an event entry form accessible to the admin only. Write a DAO method 'addCalendarEvent(CalendarEvent event)' that inserts the event into a calendar_events table. This table should include: event_id, title, event_date, program_type, location, description, and reminder_flag.

Task 9.3 — Notification Center

72. Create a panel named 'NotificationPanel' accessible to CWTS and ROTC students. Query the notifications table for the logged-in student ordered by created_date descending. Display each notification card in a scrollable JPanel with the type icon label, message, and timestamp.
73. Write a service class named 'NotificationService' with a method 'generateNotifications(int studentId)' that checks relevant conditions: attendance below threshold (generate At-Risk alert), new unread demerit (generate Demerit alert), upcoming calendar event within 3 days (generate Event alert), and engagement hours below minimum before the semester ends (generate Warning alert). For each triggered condition, insert a record into the notifications table if one does not already exist for the same event.
74. Use a javax.swing.Timer in the StudentFrame that calls NotificationService.generateNotifications() every 60 seconds and updates the notification badge count on the sidebar navigation button.

EXPECTED OUTPUT

- Admin can post announcements with audience filtering saved to MySQL.
- Students see only program-relevant announcements with read tracking.
- Calendar renders correctly and loads events from the database.
- Admin can add calendar events saved to MySQL.
- Notification center loads student-specific notifications from MySQL.
- NotificationService generates correct alerts based on student data conditions.

COMMON ISSUE: The calendar GridLayout can produce incorrect rendering if the first day of the month does not start on a Sunday or Monday. Use `Calendar.getInstance()`, call `setTime()` with the first day of the selected month, and use `get(Calendar.DAY_OF_WEEK)` to determine the offset for the first cell.

DAY
10

PHASE 4 — MODULE DEVELOPMENT

Records and Reports Module, Profile Module, and Feedback

MAIN OBJECTIVE

Build the Reports and Export Center for the Administrator, the Student Profile and Account Settings panel, and the Student Feedback and Concerns module. Complete all remaining module panels not yet built.

DETAILED TASKS

Task 10.1 — Reports and Export Module

75. Create a panel named 'ReportsPanel' accessible to the Administrator. Add a JComboBox for selecting report type (Individual Student Report, Class Attendance Report, Engagement Summary, Performance Summary, Points Standing, Demerit Summary, At-Risk Students) and a student selector JComboBox (or All Students option).
76. Write a service class named 'ReportService'. For each report type, write a method that queries the relevant MySQL tables, assembles the data into a formatted string, and returns it. Example: 'generateAttendanceReport(int studentId)' queries the attendance table and formats each record as a line with session name, date, and status.
77. Add a Preview button that displays the assembled report text in a scrollable JTextArea inside a JDialog. Add an Export button that opens a JFileChooser save dialog and writes the report text to a .txt file at the chosen location using FileWriter and BufferedWriter.

Task 10.2 — Student Profile and Account Settings

78. Create a panel named 'ProfilePanel'. Display the logged-in student's name, ID, program, year level, section, contact number, and email loaded from the students table. Add an Edit button that opens a JDialog allowing the student to update their contact number and email only. Write a DAO method 'updateStudentContact(int studentId, String contactNo, String email)' that updates the students table.
79. Add a Change Password section with a current password field, a new password field, and a confirm new password field. Validate that the current password matches the stored hash. Validate that the new password and confirm fields match. If both checks pass, update the password_hash field in the users table.

Task 10.3 — Feedback and Concerns Module

80. Create a panel named 'FeedbackStudentPanel' with a feedback type JComboBox, a subject JTextField, and a message JTextArea. Write a DAO method 'submitFeedback(Feedback feedback)' that inserts the record into a feedback table with fields: feedback_id, student_id, submitted_date, type, subject, body, status, admin_response.
81. Create a panel named 'FeedbackAdminPanel' that loads all pending feedback in a JTable. The admin can open each item, type a response, and click Resolve to update the status and admin_response fields.

EXPECTED OUTPUT

- All report types generate correct output from MySQL data and export to .txt files.
- Student profile panel displays and edits contact data correctly.
- Password change validates current password and updates the database.
- Feedback submission and admin resolution both save correctly to MySQL.

MILESTONE 3 — End of Day 10: All seven modules are now built. The system has functional panels for every feature defined in the system design. Days 11 through 14 focus on connecting all modules into one coherent application, testing, and preparing for demonstration.

DAY
11

PHASE 5 — SYSTEM INTEGRATION

System Integration: Connecting All Modules

MAIN OBJECTIVE

Wire all module panels into the MainFrame and StudentFrame using the CardLayout system. Verify that navigation between modules works correctly for all four user roles. Confirm that data changes in one module are immediately reflected when navigating to a related module.

DETAILED TASKS

Task 11.1 — Register All Panels in the Card Layout

82. In MainFrame, add each admin module panel to the CardLayout container using add(panel, panelName). The panel names should be consistent strings such as 'DASHBOARD', 'ATTENDANCE', 'ENGAGEMENT', 'PERFORMANCE', 'POINTS', 'DEMERITS', 'CALENDAR', 'ANNOUNCEMENTS', 'REPORTS', 'USERS', 'FEEDBACK_ADMIN'.
83. Wire each sidebar navigation button's ActionListener to call the CardLayout's show() method with the corresponding panel name. After show() is called, add a call to the panel's 'loadData()' method if one exists, so that data is refreshed each time the user navigates to that panel.
84. Repeat the same process for the StudentFrame, using the student-facing panel names: 'STUDENT_DASHBOARD', 'ATTENDANCE_VIEW', 'ENGAGEMENT_VIEW' or 'PERFORMANCE_VIEW' based on program, 'POINTS_VIEW', 'LEADERBOARD', 'NOTIFICATIONS', 'PROFILE', 'FEEDBACK'.

Task 11.2 — Apply Role-Based Access Control

85. In MainFrame, check the role stored in SessionManager when the frame is initialized. If the role is ROTC Senior, hide the sidebar buttons for User Management, Engagement, Reports, and any other admin-exclusive modules. Show only Dashboard, Performance entry, Demerit Assignment, Calendar, and Logout.
86. Add access validation in each admin panel's constructor. At the start of the constructor, check SessionManager.getInstance().getCurrentRole(). If the role does not have access to that panel, show a JOptionPane error and navigate back to the dashboard. This is a secondary check that prevents unauthorized access even if the UI button is somehow accessible.

Task 11.3 — Verify Cross-Module Data Consistency

87. Test the following data flow: log an attendance session as admin, then navigate to the student view and confirm the new record appears. Award points to a student as admin, then log in as that student

and confirm the balance on the dashboard and in the points history has updated correctly. Assign a demerit to a ROTC student, then log in as that student and confirm the notification appears in the notification center.

88. For each test, note any delays, incorrect data, or panels that do not refresh properly. Document all issues found in a shared bug tracker or notes document for resolution on Day 12.

EXPECTED OUTPUT

- All module panels correctly registered and navigable through the sidebar.
- Each panel refreshes its data when navigated to.
- ROTC Senior role sees only the correct subset of modules.
- Cross-module data consistency verified for attendance, points, and demerits.
- Bug list compiled from integration testing for Day 12 resolution.

COMMON ISSUE: If navigating back to the dashboard does not show updated values after a data change, confirm that the dashboard's `loadData()` method is being called each time the panel is shown, not just once at construction time.

TIP: Create a standard interface called 'RefreshablePanel' with a single 'loadData()' method. Have all module panels implement this interface. Then in the CardLayout navigation logic, cast the current panel to RefreshablePanel and call `loadData()` automatically on every panel switch.

DAY
12

PHASE 6 — TESTING AND DEBUGGING

Testing, Debugging, and Data Validation

MAIN OBJECTIVE

Run structured testing for every module using realistic test data. Fix all bugs found during integration testing on Day 11. Validate that all computed values (attendance rates, points balances, engagement totals) are mathematically correct based on the data in the database.

DETAILED TASKS

Task 12.1 — Prepare Test Data

89. Insert at least 5 student records into the students table using phpMyAdmin: 3 CWTS students and 2 ROTC students. Create user accounts linked to each student with the correct roles. Create 1 admin account and 1 ROTC Senior account.
90. Insert at least 3 attendance sessions with mixed statuses across all students. Insert at least 4 engagement activity records for one CWTS student. Insert at least 5 drill performance entries for one ROTC student. Insert points transactions totaling a non-zero balance for both a CWTS and ROTC student. Assign 2 demerits to one ROTC student.

Task 12.2 — Module-by-Module Testing

For each module, perform the following checks and record the result as Pass or Fail in a shared test log.

Module	Test Cases
--------	------------

Login	Test valid credentials for all 4 roles. Test invalid credentials. Confirm correct dashboard opens for each role.
Attendance	Log a session. Verify records appear in student view. Submit a dispute. Confirm dispute status updates after admin resolution.
Engagement	Add activities. Verify hours total is correct. Test completion badge appears when hours meet threshold.
Performance	Add 3 consecutive low ratings. Confirm at-risk flag appears. Add one high rating and confirm flag is cleared.
Points	Award and deduct points. Verify balance matches manual calculation from transaction history.
Demerits	Assign 3 demerits of different severities. Confirm weighted score is correct. Dismiss one and confirm status updates.
Announcements	Post one Urgent pinned announcement for CWTS only. Log in as CWTS student and confirm it appears at top. Log in as ROTC student and confirm it does not appear.
Calendar	Add an event 2 days from today. Log in as a student and confirm the notification alert is generated.
Reports	Generate an Individual Student Report and export to .txt. Open the file and verify the content is complete and correctly formatted.
Profile	Update contact number and confirm the change persists after re-login. Change password and confirm the new password allows login.

Task 12.3 — Fix All Identified Bugs

91. Work through the bug list compiled on Day 11 and any new issues found today. For each bug, identify the root cause (DAO query error, UI refresh issue, logic error, or database constraint violation), apply the fix, and re-test the affected module to confirm the fix works.
92. Test the logout function from all user roles. Confirm that `SessionManager.clearSession()` is called, the student or admin frame is disposed, and the login screen reappears without any data from the previous session visible.

EXPECTED OUTPUT

- Test data populated in MySQL with at least 5 students, sessions, activities, drills, points, and demerits.
- All modules pass their test cases with no critical bugs remaining.
- Computed values for attendance rate, points balance, and engagement hours verified as mathematically correct.
- Logout fully clears session and returns to login screen.

MILESTONE 4 — End of Day 12: The system is fully tested and all critical bugs are resolved. All modules produce correct data from MySQL. The remaining two days are for UI polish, final validation, and demonstration preparation.

MAIN OBJECTIVE

Apply consistent visual styling across all panels, add robust input validation to all forms, implement password hashing for the users table, and add error handling to all database operations so that the system responds gracefully to unexpected inputs and connection issues.

DETAILED TASKS**Task 13.1 — UI Consistency and Styling**

93. Create a shared class named 'UIStyle' in the util package. Define static constants for the primary colors (navy, gold, white, light gray), the standard fonts (bold heading font at 14pt, body font at 12pt), and the standard button size (120px by 35px). Use these constants in every panel to ensure visual consistency.
94. Review every panel and apply the following consistency rules: all form labels are right-aligned, all input fields are the same width within each form, all action buttons are bottom-aligned within their containing panel, all JTables have the same row height (25px) and a consistent header background color, and all error dialogs use JOptionPane.ERROR_MESSAGE with a descriptive message.
95. Confirm that the application title bar shows 'NSTP Information System' on every window and dialog. Set the application icon using `frame.setIconImage()` with a small logo image placed in the resources folder.

Task 13.2 — Input Validation

96. Add input validation to every form in the system. For text fields, check that required fields are not empty before allowing the save action. For numeric fields, verify that the input can be parsed as a number and is within an acceptable range. For date fields, confirm the date is not in the future for past-event entries. For the hours field in engagement, confirm the value is greater than 0 and not greater than 24.
97. For the login form, trim whitespace from the username field before submitting. For the password change form, check that the new password is at least 8 characters and contains at least one letter and one number.

Task 13.3 — Password Hashing

98. Replace plain text password storage with SHA-256 hashing. Write a utility method in the util package named `hashPassword(String plainText)` that uses `MessageDigest` with the SHA-256 algorithm and returns a hexadecimal string.
99. Update `UserDAO.validateLogin()` to hash the input password before comparing it to the stored hash. Update the password change logic in `ProfilePanel` to hash the new password before saving. Update any existing plain-text passwords in the test data using `phpMyAdmin` or a one-time fix script.

Task 13.4 — Database Error Handling

100. Wrap all DAO method calls in try-catch blocks that catch `SQLException`. In the catch block, print the full stack trace to the console for debugging and show a user-friendly `JOptionPane` error dialog that says 'A database error occurred. Please try again or contact your administrator.' Do not show the raw SQL error message to the user.

101. Add a connection check at the start of each DAO method. If `DBConnection.getConnection()` returns null, throw a custom exception or show an immediate error dialog informing the user that the system cannot connect to the database and that they should confirm MySQL is running.

EXPECTED OUTPUT

- All panels follow the same visual style defined in `UIStyle`.
- All forms validate input before saving and show clear error messages.
- Passwords are stored as SHA-256 hashes in the database.
- All DAO methods handle `SQLException` gracefully with user-friendly messages.
- Database connection failure is detected early and reported to the user.

COMMON ISSUE: If applying a global font in `UIManager` seems to have no effect on `JTables` or `JComboBoxes`, apply the font directly to the component using `component.setFont()` after creation. `UIManager` defaults are sometimes overridden by component look-and-feel settings.

TIP: SHA-256 is sufficient for an academic prototype. In a production system, use `BCrypt` with a salt. Note this limitation in your research documentation if asked during the defense.

DAY

14

PHASE 7 — FINALIZATION

Final System Testing, Demo Data, and Demonstration Preparation

MAIN OBJECTIVE

Conduct a complete end-to-end walkthrough of the entire system, prepare realistic demonstration data in the database, create a startup checklist for the demonstration day, and confirm that every team member can run, explain, and demonstrate their assigned module confidently.

DETAILED TASKS

Task 14.1 — Final End-to-End System Test

102. Run through the complete user journey for each role without stopping. For the Administrator: log in, view dashboard, log a full attendance session, post an announcement, generate an individual student report, and export it. For the CWTS Student: log in, view dashboard, check attendance records, view engagement hours, check notification center. For the ROTC Student: log in, view dashboard, check performance history, check demerit records, view notification. For the ROTC Senior Officer: log in, submit a drill rating, assign one demerit, log out.
103. Document any visual issues, missing data, incorrect computations, or UI elements that do not respond as expected. Fix all issues before marking the system as ready for demonstration.

Task 14.2 — Prepare Demonstration Database

104. Clean the test database and insert polished demonstration data. Use realistic Filipino names and PLM-appropriate section names and year levels. Ensure that the data tells a clear story: one CWTS student who has completed their engagement hours, one ROTC student who is at-risk due to low performance ratings, and one student with an unresolved attendance dispute.
105. Export the complete demonstration database as a .sql file named 'nstp_demo_database.sql'. Store this file in the project folder and on a USB drive. This file allows the team to restore the demonstration data quickly if the database is accidentally modified during the demonstration.

Task 14.3 — Create the System Startup Checklist

Prepare a printed or saved checklist for the demonstration day. The checklist must include the following items in order.

106. Power on the machine and confirm it is the one with XAMPP and the Java project installed.
107. Open XAMPP Control Panel and start both the Apache and MySQL services. Wait for green indicators before proceeding.
108. Open phpMyAdmin and confirm the 'nstp_infosystem' database is present and contains data.
109. Open the IDE (NetBeans or IntelliJ) and load the NSTP_InfoSystem project.
110. Run the project. Confirm the login screen appears without errors.
111. Log in as the admin account using the demonstration credentials.
112. Navigate through two or three modules to confirm everything is working before the panel or instructor arrives.
113. If any issue is found, restore the database from 'nstp_demo_database.sql' using phpMyAdmin Import.

Task 14.4 — Assign Demonstration Roles to Team Members

114. Divide the system into sections and assign each team member one or two modules to explain and demonstrate. Each team member should be able to: explain what the module does, show the data being saved and retrieved from MySQL, and answer at least one question about the logic behind the feature.
115. Conduct one internal dry-run demonstration. One team member acts as the instructor and asks questions. The others demonstrate and answer as they would during the actual defense or consultation. Note any questions that the team cannot answer confidently and review those topics before the actual demonstration.

EXPECTED OUTPUT

- Full end-to-end walkthrough completed with all critical issues resolved.
- Demonstration database prepared with realistic and clear test data.
- Database backup file 'nstp_demo_database.sql' saved and accessible on USB and project folder.
- Printed or saved system startup checklist ready for demonstration day.
- Each team member knows their assigned module and can explain and demonstrate it.
- Internal dry-run demonstration completed and all anticipated questions reviewed.

FINAL MILESTONE — End of Day 14: The NSTP Integrated Information System is complete. All modules are functional, the MySQL database is populated with demonstration data, the team is prepared to demonstrate the system, and a recovery plan is in place.

TIP: Arrive early on demonstration day, start XAMPP and verify the system works before any evaluator is present. Having the system already running and displaying the login screen when the demonstration begins shows confidence and preparation.

Appendix: Quick Reference Summary

14-Day Plan at a Glance

Day	Focus and Key Deliverables
Day 1	Install JDK 21, NetBeans/IntelliJ, XAMPP, MySQL Connector. Create Java project. Set up version control.
Day 2	Create MySQL database 'nstp_infosystem'. Define all 10 tables. Build DBConnection class. Test JDBC connection.
Day 3	Define package structure. Build SessionManager (Singleton). Create all model classes. Create Main entry point.
Day 4	Build LoginFrame with UserDAO authentication. Build MainFrame and StudentFrame shells with CardLayout sidebar.
Day 5	Build User Management admin panel (JTable, add/edit/deactivate). Build Student Dashboard with summary cards.
Day 6	Build Attendance admin logging panel (bulk entry, duplicate guard). Build student attendance view with disputes.
Day 7	Build Engagement module (CWTS hours, completion). Build Performance module (ROTC ratings, at-risk detection).
Day 8	Build Points module (award, deduct, balance, leaderboard). Build Demerit module (assign, dismiss, student view).
Day 9	Build Announcements (audience filtering, read tracking). Build Calendar (event dots, event entry). Build Notification Center with auto-generation service.
Day 10	Build Reports and Export module. Build Profile and password change panel. Build Feedback and Concerns module.
Day 11	Wire all panels into CardLayout. Apply RBAC per role. Test cross-module data consistency. Compile bug list.
Day 12	Insert structured test data. Run module-by-module tests. Fix all bugs. Verify all computed values.
Day 13	Apply UIStyle constants across all panels. Add input validation. Implement SHA-256 password hashing. Add database error handling.
Day 14	Final end-to-end walkthrough. Prepare demo database and SQL backup. Create startup checklist. Assign demo roles. Conduct dry run.

Common Warnings Summary

Warning	Solution
JDBC Connection Failure	Confirm XAMPP MySQL is running. Verify port 3306. Check database name spelling in JDBC URL.
GUI Not Refreshing	Call loadData() on every panel navigation. Avoid caching panel data only at construction time.
JTable Not Updating	Call tableModel.fireTableDataChanged() after any add, edit, or delete operation on the model.

Duplicate Session Data	Always check for existing records before INSERT. Use a query that counts matching session name and date.
Password Mismatch After Hash	Confirm the same hashing algorithm is used at both save time and login comparison time.
GUI Layout Broken	Call pack() or setSize() after all components are added. Always set visible inside SwingUtilities.invokeLater().
Calendar Day Offset Error	Use Calendar.DAY_OF_WEEK to find the correct starting column for the first day of each month.
Performance Slow on Load	Use SQL-level aggregation (SUM, AVG, COUNT) instead of fetching all rows into Java and computing in a loop.